

Backend Architecture

Date updated: 2026-06-03

Purpose

The backend is the API and database layer for the secure messaging system. It authenticates users, validates requests, stores public key material, relays encrypted direct 1:1 message payloads, enforces message-level authorization, records audit events, creates blockchain fidelity anchor metadata, and exposes status/verification endpoints for those anchors.

Backend boundaries:

- Message plaintext stays on client devices.
- Long-term private keys stay on client devices.
- Signal/libsignal session state stays on client devices.
- FastAPI request handlers create pending blockchain anchors but do not hold wallet credentials or submit Sepolia transactions.
- The backend blockchain worker is the deployed process that reads pending anchors and submits them to Sepolia.

The architecture is deliberately split into small layers so each assessed behaviour is easy to defend in the demo/interview: routes define the HTTP contract, dependencies enforce shared security checks, services own business rules and transactions, repositories own SQLAlchemy queries, and models/migrations define the database state.

Assessed Backend Responsibilities

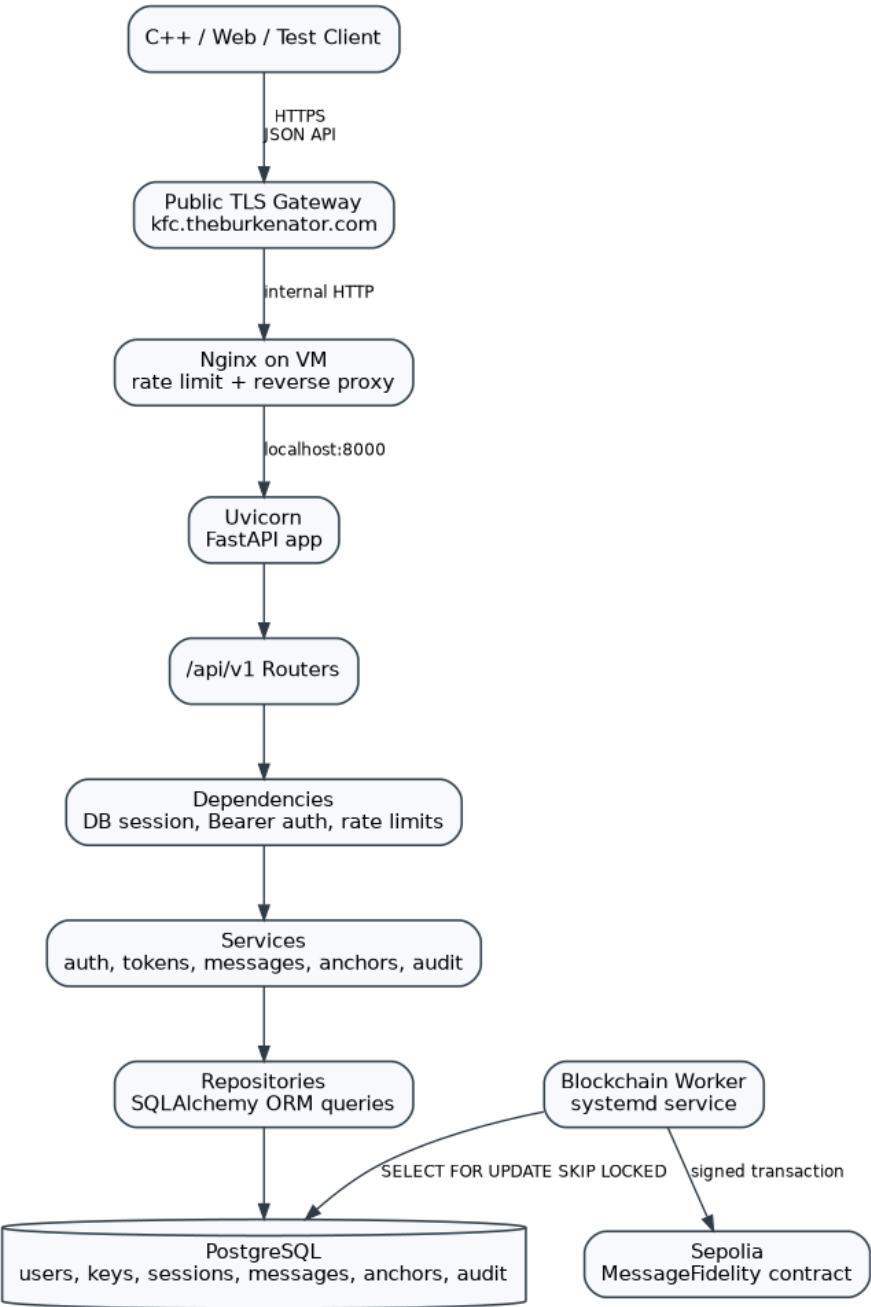
Responsibility from brief	Backend implementation
Backend service accepts and processes requests	FastAPI <code>/api/v1</code> routes for auth, users, keys, messages, and blockchain anchors.
Secure client/server connectivity	Public HTTPS through the TLS gateway, Nginx proxy on VM, localhost Uvicorn, FastAPI HTTPS/HSTS controls.
Server-side authentication	Argon2id password hashes, signed JWT access tokens, HMAC-hashed refresh tokens, refresh rotation, logout revocation.
Server-side authorization	<code>get_current_user()</code> dependency and service-level object checks for message/key/anchor access.
Secure database use	Async SQLAlchemy ORM with PostgreSQL, migrations, repository layer, no string-built request SQL.
Input validation	Strict Pydantic schemas, UUID/positive ID/base64/wire-payload validation, sanitized validation errors.
Sensitive data protection	No plaintext/private-key storage; response schemas omit password and refresh-session hashes; audit detail allowlist.
Blockchain fidelity proof	Pending anchors from encrypted message metadata; worker submission to <code>MessageFidelity.storeHash</code> on Sepolia.
Vulnerability evidence	Unit, integration, security, Bandit, pip-audit, and DB-backed test results recorded in security docs.

Implementation Evidence Map

Area	Main implementation files	Test/evidence files
App startup, CORS, headers, HTTPS enforcement, validation sanitization	backend/app/main.py, backend/app/core/config.py	backend/tests/integration/test_security_headers.py
Authentication and sessions	backend/app/api/v1/auth.py, backend/app/services/auth_service.py, backend/app/services/password_service.py, backend/app/services/token_service.py	backend/tests/security/test_auth_security.py, backend/tests/integration/test_auth_routes.py, backend/tests/unit/test_password_service.py, backend/tests/unit/test_token_service.py
Current-user dependency and rate limits	backend/app/api/deps.py, backend/app/core/rate_limit.py	backend/tests/security/test_rate_limit_security.py, backend/tests/integration/test_rate_limiting.py
Public key relay	backend/app/api/v1/keys.py, backend/app/schemas/device_key.py, backend/app/schemas/one_time_prekey.py, key repositories	backend/tests/integration/test_key_routes.py, backend/tests/integration/test_key_repositories.py

Area	Main implementation files	Test/evidence files
Message relay and object-level access control	backend/app/api/v1/messages.py, backend/app/services/ message_service.py, backend/app/repositories/ message_repository.py	backend/tests/security/ test_access_control_security.py, backend/tests/integration/ test_message_routes.py, backend/tests/integration/ test_message_service.py
Blockchain anchor metadata and worker	backend/app/api/v1/blockchain.py, backend/app/services/ blockchain_anchor_service.py, backend/app/core/ blockchain_hashing.py, backend/app/workers/ blockchain_worker.py	backend/tests/integration/ test_blockchain_routes.py, backend/tests/unit/ test_blockchain_anchor_service.py
Sensitive-data minimization and audit logging	backend/app/services/ audit_service.py, response schemas, validation handler	backend/tests/security/ test_sensitive_data_security.py, backend/tests/integration/ test_audit_logging.py
Database schema	backend/app/models/*.py, backend/alembic/ versions/*.py	migration checks in docs/security/ security_test_results.md

Runtime Topology



Runtime Processes

Process	File/config	Role
FastAPI app	<code>backend/app/main.py</code>	Creates app, installs CORS, security headers, HTTPS enforcement, validation sanitization, and <code>/api/v1</code> routing.
Uvicorn API service	<code>backend/deploy/epic-messaging-api.service</code>	Runs the API on the VM, bound behind Nginx.
Nginx reverse proxy	<code>backend/deploy/nginx/epic-messaging-api.conf</code>	Receives gateway traffic, applies public-edge rate limits, forwards to Uvicorn.
PostgreSQL	Deployment/runbook	Stores application data and migration-created schema.
Blockchain worker	<code>backend/deploy/epic-messaging-blockchain-worker.service</code>	Runs separately from FastAPI, reads pending anchor rows, submits contract transactions, records confirmations.

API Module Ownership

Router	Prefix	Responsibility	Security rules
<code>auth.py</code>	<code>/api/v1/auth</code>	Register, login, refresh, logout, current profile	IP rate limits, Argon2id, generic failures, JWT access tokens, HMAC refresh-token hashes
<code>users.py</code>	<code>/api/v1/users</code>	Authenticated user discovery by username	Bearer auth; public-safe user/device summary only
<code>keys.py</code>	<code>/api/v1/keys</code>	Public device keys and one-time prekey bundles	Bearer auth; current-user ownership for upload; public key fields only; one-time prekey row locking
<code>messages.py</code>	<code>/api/v1/messages</code>	Direct encrypted message relay, list, fetch, forward, revoke, delete, message anchor status	Bearer auth; sender from JWT only; sender/recipient visibility checks; per-user rate limits
<code>blockchain.py</code>	<code>/api/v1/blockchain</code>	Anchor create/reuse, anchor status, backend proof verification	Bearer auth; linked-message access check; verify compares confirmed DB metadata

Code Layering

HTTP request

- > FastAPI router
- > route dependencies
- > service workflow
- > repository query/persistence
- > AsyncSession transaction
- > PostgreSQL tables

Layer	Files	Responsibility
App setup	<code>backend/app/main.py</code>	CORS, HTTPS/security headers, validation sanitization, router registration.
API routes	<code>backend/app/api/v1/*.py</code>	HTTP paths, status codes, request/response models, dependency wiring.
Dependencies	<code>backend/app/api/deps.py</code>	Per-request DB session, Bearer token parsing, active-user loading, rate-limit checks.
Services	<code>backend/app/services/*.py</code>	Business rules, transactions, authorization, audit calls, token workflows, anchor creation.
Repositories	<code>backend/app/repositories/*.py</code>	SQLAlchemy query construction and persistence operations.
Schemas	<code>backend/app/schemas/*.py</code>	Pydantic validation and serialization boundaries.

Layer	Files	Responsibility
Models	<code>backend/app/models/*.py</code>	PostgreSQL tables mapped through SQLAlchemy ORM.
Migrations	<code>backend/alembic/versions/*.py</code>	Schema creation and evolution.
Worker	<code>backend/app/workers/blockchain_worker.py</code>	Sepolia contract submission from pending anchor rows.
Tests	<code>backend/tests/*</code>	Unit, integration, and security evidence.

HTTP Request Lifecycle

Every API request follows the same security order:

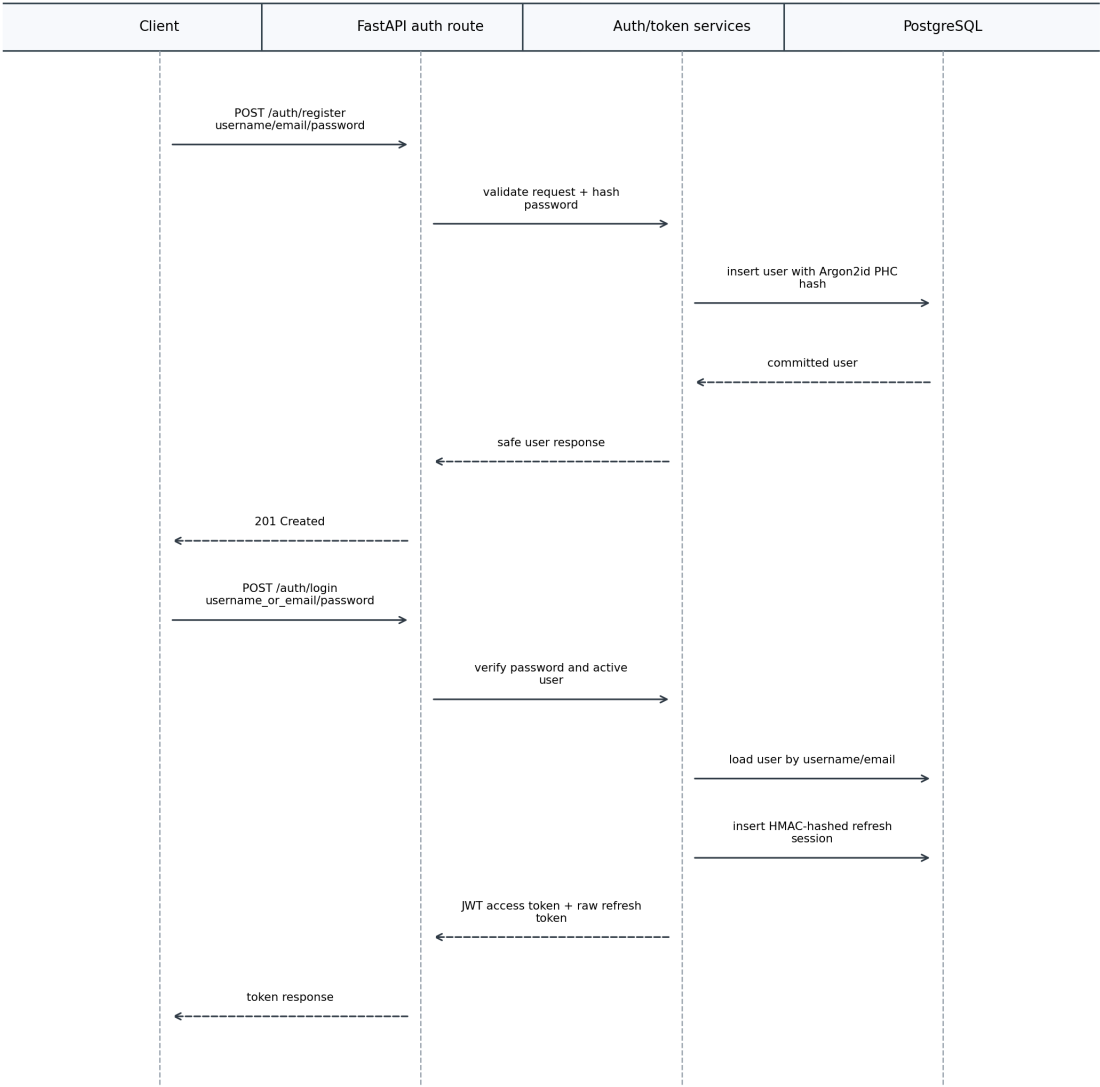
1. The client sends JSON over HTTPS to the public hostname.
2. The TLS gateway terminates the public certificate and forwards traffic to the VM.
3. Nginx applies edge request limits and proxies to local Uvicorn.
4. FastAPI middleware enforces HTTPS semantics when configured, adds security headers, and sanitizes validation errors.
5. Route dependencies create an `AsyncSession`, parse Bearer credentials when required, and enforce route-specific rate limits.
6. Pydantic validates body, path, and query input before service logic runs.
7. Service functions apply business rules and object-level authorization.
8. Repository functions build SQLAlchemy ORM queries against PostgreSQL.
9. The service commits or rolls back the transaction.
10. Response schemas serialize only the fields intended for the caller.

This order matters because invalid input is rejected before DB workflows, authentication happens before protected service logic, and object authorization happens before message or anchor records are returned.

Data Ownership Boundaries

Data	Owner/source of truth	Backend handling
Account username/ email/password	User and auth service	Password is hashed with Argon2id; email is not exposed by public discovery
Access-token identity	Token service	JWT subject is the authenticated user ID used by protected routes
Refresh token	Client holds raw value; backend stores HMAC hash	Refresh rotates under row lock and revokes the old hash
Device public keys	Client-generated public material	Backend validates base64 and stores public fields only
Private keys and ratchet state	Client only	No schema/model columns exist for these values
Plaintext message body	Client only	Not accepted by message routes; rejected as unexpected input
wire_payload_json	Client encrypted payload	Backend validates structure/size and stores as opaque ciphertext
Message visibility	Backend service	Sender/recipient visibility timestamps control list/ fetch behaviour
Blockchain record ID and digest	Backend anchor service	Derived from message ID and canonical encrypted metadata
Sepolia transaction metadata	Blockchain worker	Worker writes tx hash, contract address, status, and anchored timestamp

Authentication Lifecycle



Sequence diagram

Important details:

- Duplicate registration and login errors are generic.
- Password hashes are Argon2id PHC strings.
- Access tokens contain required signed claims and `type=access`.
- Refresh-token DB rows contain HMAC hashes, not raw refresh tokens.

- Refresh rotation revokes the old session and creates a new session under row lock.
- Logout revokes the submitted refresh token without confirming whether it existed.

Evidence: `backend/app/services/auth_service.py`, `backend/app/services/password_service.py`, `backend/app/services/token_service.py`, `backend/app/api/deps.py`, auth tests.

Authorization Model

The backend uses two authorization layers:

1. Route-level authentication loads an active current user from a verified Bearer access token.
2. Service/repository object checks decide whether that user can perform the requested action.

Operation	Authorization rule
Upload device key	Current user owns the device key row being created/updated.
Upload one-time prekeys	Current user owns an active device with the path <code>device_id</code> .
Fetch prekey bundle	Requester is authenticated; target user/device must be active.
Send message	Sender is current user; sender device belongs to sender; recipient/device active.
List inbox	Current user is recipient and recipient copy is visible.
List sent	Current user is sender and sender copy is visible.
Fetch message	Current user is sender or visible recipient.
Forward message	Current user can access original message; new message sender is current user.
Revoke message	Current user is original sender.

Operation

Delete message

Fetch anchor

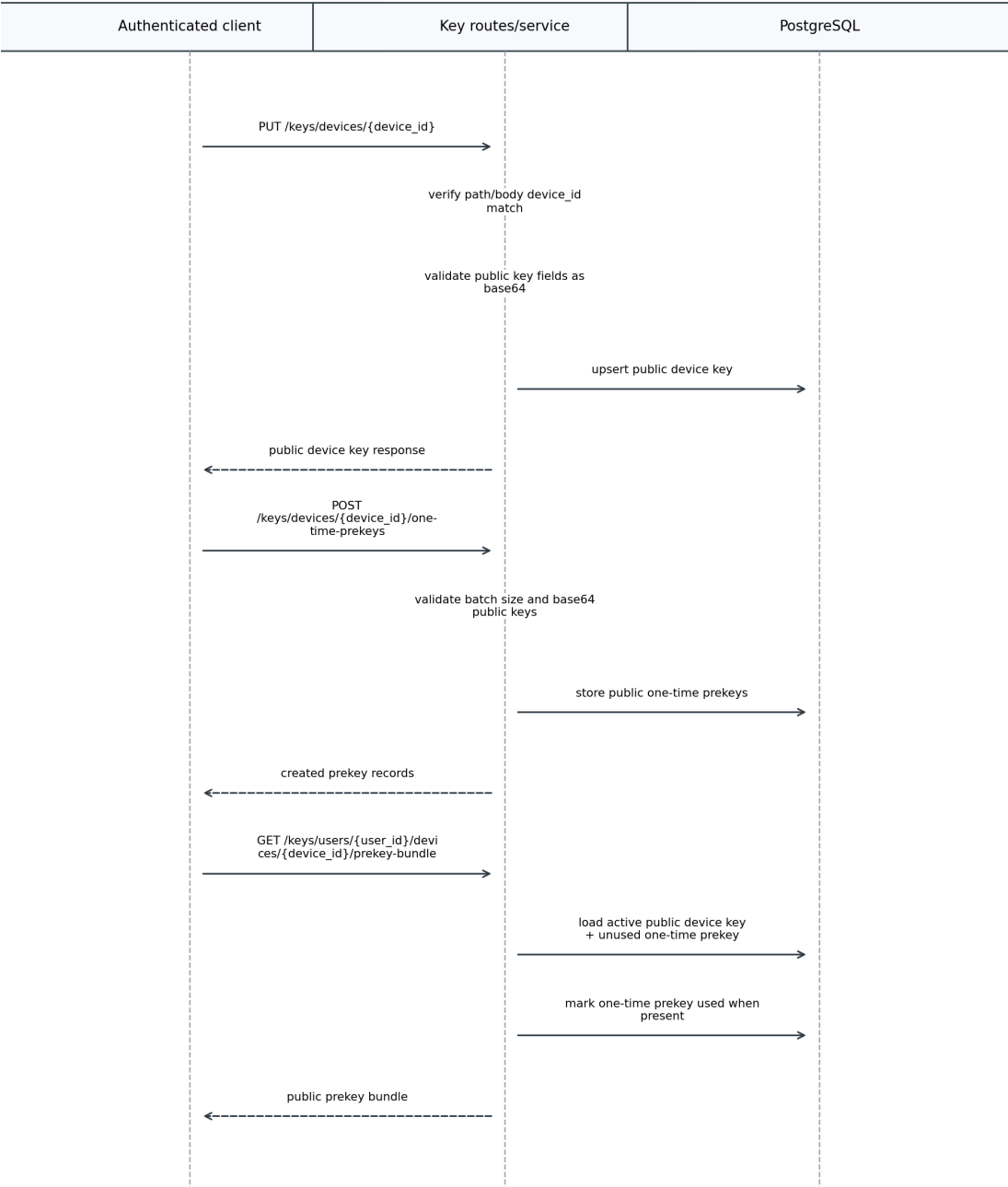
Authorization rule

Current user is sender or recipient;
only that user's visibility field is set.

Current user can access linked
message.

The API returns safe not-found errors for inaccessible messages and anchors, so another user's object existence is not disclosed.

Public Key Relay Lifecycle



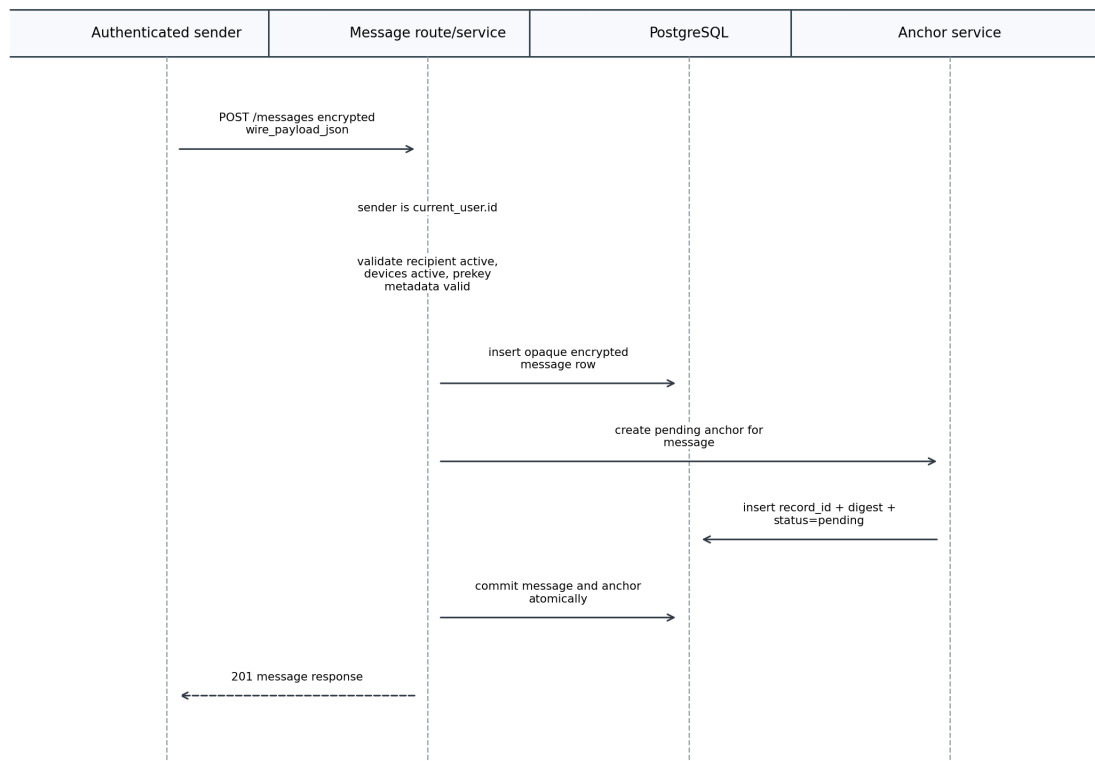
Sequence diagram

Security properties:

- Only authenticated users upload keys.
- Uploaded key material is public key/signature material only.
- Private-key fields are not modeled or stored.
- One-time prekey consumption is recorded in the backend database.
- The backend cannot prove the encrypted payload used the consumed prekey; the client cryptography deliverable proves protocol correctness.
- Public key upload and prekey fetch are relay operations only; the backend does not assert long-term identity trust. TOFU/key-pinning behaviour belongs to the client/cryptography component.

Evidence: `backend/app/api/v1/keys.py`, `backend/app/schemas/device_key.py`, `backend/app/schemas/one_time_prekey.py`, `key route/repository` tests.

Message Send Lifecycle



Sequence diagram

Security properties:

- Request body cannot set `sender_user_id`.
- Recipient user and sender/recipient devices must be active.
- Optional `consumed_one_time_prekey_id` must match a recipient/device prekey that was already consumed by bundle fetch.
- `wire_payload_json` is validated for structure and size, then stored as opaque encrypted data.
- The backend does not decrypt or inspect plaintext.
- A pending anchor is created in the same DB transaction as the message.

Evidence: `backend/app/schemas/message.py`, `backend/app/services/message_service.py`, `backend/app/repositories/message_repository.py`, `backend/app/services/blockchain_anchor_service.py`, `message` and `blockchain` tests.

Transaction behaviour:

- `_validate_send_inputs()` runs before message insertion.
- Message row and pending blockchain anchor are created in one transaction.
- Any validation or persistence error rolls back both the message and anchor.
- The returned message is refreshed from PostgreSQL after commit.

Message Read, Forward, Revoke, And Delete Lifecycles

Read And List

- Inbox queries return messages visible to the authenticated recipient.
- Sent queries return messages visible to the authenticated sender.
- Direct message fetch uses sender/recipient visibility checks.
- Inaccessible messages return safe not-found responses.

Forward

1. The user requests a forward for an existing message ID.
2. The service loads the original message only if visible to the current user.
3. The request supplies a new encrypted `wire_payload_json` for the new recipient.
4. The service creates a new message row with server-controlled `forwarded_from_message_id`.
5. A new pending blockchain anchor is created for the forwarded message.

Forwarding preserves auditable provenance without decrypting or copying plaintext.

Revoke

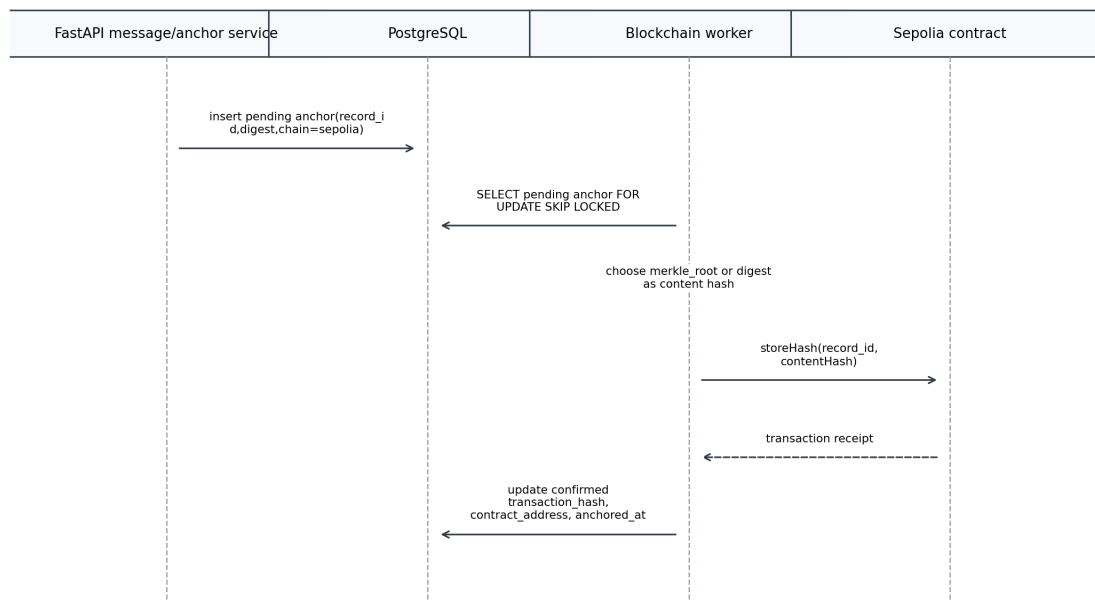
- Only the sender can revoke recipient access.
- Revocation sets `access_revoked_at`.
- Recipient reads stop returning the revoked message.

Delete

- Sender delete sets sender visibility state.
- Recipient delete sets recipient visibility state.
- Delete is not an immediate physical deletion because auditability and sender/recipient visibility are separate.

Evidence: `backend/app/services/message_service.py`, `backend/app/repositories/message_repository.py`, message route/security tests.

Blockchain Anchor Lifecycle



Sequence diagram

Backend hashing:

- `derive_message_record_id(message_id)` returns `keccak256("message:" + message_id)`.

- `derive_message_digest(message)` returns Keccak over canonical JSON containing message ID, sender/recipient IDs, device IDs, timestamp, encrypted `wire_payload_json`, and forwarding lineage.
- Digest input excludes plaintext, private keys, and client ratchet/session state.

Worker behaviour:

- The worker imports web3 lazily so normal API imports do not require blockchain runtime use.
- `MessageFidelitySubmitter.from_settings()` requires Sepolia RPC URL, worker wallet private key, contract address, chain ID, gas limit, and receipt timeout.
- Pending rows are locked with `FOR UPDATE SKIP LOCKED` to prevent duplicate submissions by parallel workers.
- Successful receipts update `status="confirmed"`, `transaction_hash`, `contract_address`, `merkle_root`, and `anchored_at`.
- Malformed anchors are marked failed; transient submission errors roll back unless configured to mark failed.

Production deployment:

- The production backend runs the blockchain worker as a separate service with required environment variables.
- FastAPI keeps wallet credentials out of request handlers.
- Sepolia/RPC/worker outages leave anchors pending; persisted pending rows are processed when the worker resumes.

Evidence: `backend/app/core/blockchain_hashing.py`, `backend/app/services/blockchain_anchor_service.py`, `backend/app/workers/blockchain_worker.py`, `backend/deploy/epic-messaging-blockchain-worker.service`, blockchain tests.

Failure Handling

Failure	API behaviour	Reason
Invalid JSON/schema	422 sanitized validation response	Rejects malformed input before service logic
Duplicate user	409 Username or email is unavailable	Avoids telling whether username or email matched
Bad login	401 Invalid credentials	Avoids account enumeration
Bad Bearer token	401 Invalid authentication credentials	Covers missing, invalid, expired, inactive-user, wrong-type tokens
Inaccessible message	404 Message not found	Avoids object existence disclosure
Inaccessible anchor	404 Anchor not found	Avoids proof metadata disclosure
Invalid recipient/device/prekey workflow	400 Message could not be processed	Valid JSON, invalid backend workflow state
Rate limit exceeded	429 Too many requests	Stops repeated abuse at API level
Worker/RPC outage	Anchor remains pending	Message API is not blocked by Sepolia availability

Security Design Rationale

- Keep FastAPI request handlers short and predictable, with security decisions in dependencies and services.
- Keep database queries in repositories so injection review has one obvious place to inspect.
- Store only ciphertext and public key material so an honest-but-curious backend cannot read message content.

- Use server-side visibility timestamps rather than hard deletion so sender/recipient views can be controlled independently.
- Create blockchain anchors automatically on send/forward so integrity evidence is part of the normal message workflow.
- Run Sepolia submission in a worker so wallet credentials are not present in request handlers and message send latency is not tied to chain confirmation.

Database Model Summary

Table/model	Purpose	Security notes
users	Account identity and password hash	Stores Argon2id password hash, not plaintext password.
refresh_sessions	Refresh-token session state	Stores HMAC hash, JTI, expiry, revocation time, IP/user-agent metadata.
device_keys	Public device key bundle	Stores public identity/signing/signed-prekey material only.
one_time_prekeys	Public one-time prekeys	Tracks consumption with used_at.
messages	Opaque encrypted direct messages	Stores ciphertext payload and routing/visibility metadata.
blockchain_anchors	Fidelity proof metadata	Stores record ID, digest/root, chain, status, tx hash, contract, anchor time.
audit_logs	Security/audit events	Stores allowlisted event details.

Migrations:

- 20260527_0001_create_initial_secure_messaging_schema.py
- 20260602_0002_extend_blockchain_anchor_metadata.py

- `20260602_0003_add_message_forwarding_lineage.py`

Database Access Pattern

- `backend/app/db/session.py` creates async SQLAlchemy engine/session handling.
- Each request receives an `AsyncSession` from the dependency layer.
- Services own commits and rollbacks.
- Repositories build ORM statements and return models.
- Refresh rotation and blockchain worker queue processing use row locking for concurrency-sensitive workflows.
- The application expects a `postgresql+asyncpg://` database URL.

Repository pattern:

- Routes do not build SQL.
- Services call repository functions and own transaction boundaries.
- Repositories use SQLAlchemy expressions rather than string concatenation.
- Tests exercise repository and service behaviour against the DB-backed test database.

Security Controls In The Architecture

Control	Where it sits
TLS/public certificate	Public gateway and client validation path
Edge request limit	Nginx
HTTPS/HSTS/security headers	FastAPI middleware
CORS policy	FastAPI CORS setup and settings validation
Request validation	Pydantic schemas before service logic
Authentication	<code>get_current_user()</code> dependency and token service
Authorization	Service/repository object predicates
Password hashing	Auth/password service
Refresh-token replay control	Auth service and refresh-session repository
Injection prevention	SQLAlchemy repositories
Sensitive data minimization	Schemas, audit service, validation sanitization
Blockchain fidelity	Message service, anchor service, hashing helpers, worker

Validation Evidence

The final DB-backed evidence run is recorded in `docs/security/security_test_results.md`. It includes:

- Alembic current database revision check.
- Ruff linting.
- Python compile check.
- Bandit static analysis.
- pip-audit scans for the virtualenv and `requirements.txt`.

- Unit, integration, and security tests with PostgreSQL.

Explicit Non-Goals

- Plaintext message storage.
- Message decryption.
- Private key storage.
- Server-side Signal ratchet/session state.
- Group chat or conversation model.
- Direct Ethereum transaction submission from FastAPI request handlers.
- Admin audit-log viewer.
- Distributed rate limiter.
- MFA.